This returns a WordListCorpusReader object and can be used to identify stop words which are in most cases a well-accepted list of words. This object can be used as follows:

```
if word not in stopWords:
            print(word)
```

Sentence categorisation

Sentence categorisation is a heuristic approach and depends on the application areas. A very simplified approach is used here to assign a weightage to each sentence depending on the frequency of words within the given text.

```
j = n # can be selected judicially
def score (sentences, freqTable) -> dict:
    sentenceScore = dict()

    for sentence in sentences:
        #word_count_in_sentence = (len(word_
tokenize(sentence)))
word_count_in_sentence = (len(word_tokenize(sentence)))
for wordValue in freqTable:
            if wordValue in sentence.lower():
                if sentence[:j] in sentenceScore:
                    sentenceScore[sentence[:j]] +=
freqTable[wordValue]
                else:
                    sentenceScore[sentence[:j]] =
freqTable[wordValue]
        #Normalize sentence score
        sentenceScore[sentence[:j]] =
sentenceScore[sentence[:j]] // word_count_in_sentence

 return sentenceScore
```

The global variable *j* extracts a string from each sentence to assign the cumulative score value to each *sentenceScore[]* entry, where the *sentenceScore[]* is a dictionary to represent the score of each sentence.

Summarisation

To summarise a text body, we need a threshold value to identify the most meaningful sentences to represent the text body. For that, the average of sentence scores is considered as the threshold, and the summary of the text-body is generated by the selection of all those sentences for which their score is more than the threshold value.

```
def averageScore(sentenceScore) -> int:
    sumScores = 0
    for entry in sentenceScore :
        sumScores += sentenceScore[entry]

    # Average value of a sentence score from original text
    average = (sumScores // len(sentenceScore))

    return average

def calculateSummary(sentences, sentenceScore, threshold):
    sentenceCount = 0
    summary = ''
    for sentence in sentences:
        if sentence[:j] in sentenceScore:
            if sentenceScore[sentence[:j]] > threshold:
               summary += " " + sentence
               sentenceCount += 1

    return [summary, sentenceCount]
```

Finally, all these can be used to summarise a given text.

```
textBody = """

"Natural Language Processing (NLP) is a subfield of artificial
intelligence. NLP helps computers understand and process
human language. Tasks like text summarisation, sentiment
analysis, and machine translation are part of NLP."
"""

[freqTable,wordCount] = createFrequencyTable(textBody)
sentences = sent_Tokenize(textBody)
j = 8 # an arbitrary value can be adjusted as per need
sentenceScore = score(sentences, freqTable)
# Find the threshold
threshold = averageScore(sentenceValue)
# Generate the summary
[summary, sentenceCount] = calculateSummary(sentences,
sentenceScore,  threshold)
print(summary)
print(sentenceCount)
```

The output will be tasks like text summarisation, sentiment analysis, and machine translation.

Frequency-based summarisation provides a straightforward and efficient baseline for quick overviews or in settings with limited resources. Other summarising methods, such as abstractive and hybrid, may yield superior outcomes, but they necessitate a thorough understanding of AI and ML in addition to language concepts. END

By: Dr Dipankar Ray

The author has more than 30 years of experience in software development. He has published two books on MATLAB and medical image processing (published by PHI Learning and SPD).